

Contracts and virtual functions for the Contracts MVP

Abstract

There have been various disagreements and different understandings on how contracts on virtual function overrides should behave. There is disagreement over whether an override should allow only contracts that are identical to the functions overridden, and there's disagreement over whether requiring an override of multiple base functions to have "the same list of contract annotations" is the right choice.

So, to decouple this problem from the progress of the MVP, this paper proposes a simple solution that makes the uncertain cases ill-formed, and makes well-formed the cases that have no future compatibility risk. Thus:

1. a virtual function that overrides a base function cannot have any contract annotation on the declaration of the override. It will inherit the contract of the function that it overrides, so when called via a reference/pointer to a base, the preconditions and postconditions of the base function are still checked, but the definition that is called is the override.
2. if a virtual function overrides multiple base functions, none of those base functions can have any contract annotations on their declarations.

Rationale

The main goal here is to avoid any and all complexity, and be able to ship the MVP in C++26 without getting bogged down into questions and discussions about what "identical" means, what "narrower" means, and what "wider" means, and even what "substitutable" means, and according to what definition/principle/rule.

What the MVP currently says

In [P2388R4](#), the wording part says two relevant things:

1. If an overriding function specifies contract annotations (9.12.4), it shall specify the same list of contract annotations as its overridden functions
2. If a function overrides more than one function, all of the overridden functions shall have the same list of contract annotations

In [P2521R4](#), however, [{pro.vir}](#) says what the first part of this proposal says, that an override cannot have a contract of its own, but inherits the contract of the overridden base function.

In [{pro.2bs}](#), that same paper suggests that what happens with an override that overrides multiple base functions is something that "we do not want to specify".

What this proposal proposes saying

1. An overriding function shall not specify contract annotations, and inherits the contract of the overridden function
2. If a function overrides more than one function, none of the overridden functions shall specify contract annotations

Some explanation of what is complex and why

The MVP already has some complex bits in it for simple overrides; it states that "the program is ill formed, no diagnostic required, if name lookup in the predicate finds different entities than if the name lookup were performed in the context of the base class". This is skating around a problem with "identical" and "shall have the same list of contract annotations".

Token-by-token equality ends up being useless, because using-declarations can change what tokens mean in different scopes, and overriding functions can be in very different scopes from their base functions; they can be in different namespaces. Any use of something like `decltype(*this)` might well be token-by-token identical, but have a different meaning. IFNDR is a big hammer, and hard to understand, so we can apply a much simpler approach by completely avoiding the problematic cases.

"ODR-identical" is a lofty notion, but even if an implementation could perhaps compute that identity without much trouble, we don't have any field experience on what that's like for users. So, again, we can apply a much simpler approach by completely avoiding the problematic cases.

We actually do have an implementation of checking odr-identity for a virtual function override that has its own contract declaration; that is properly checked by GCC for identity with a single overridden base function. That implementation, however, doesn't check multiple base function contracts for identity in the case where the override doesn't have a contract declared.

So, what do we lose?

For the inability to be able to repeat the contract of an overridden function, we can't, for example, write a class with a dependent base that adds a contract if a virtual function is not an override, but repeats it if the virtual function is an override, depending on whether the dependent base declares such a virtual function.

For the same restriction, we can't repeat a contract that we'd like to repeat for ease-of-reading purposes, or for the hope that if the contract ends up deviating from the base function's contract, we'd get some sort of a diagnostic.

For the multiple inheritance restriction, we can't mix base classes with virtual functions with contracts on them into classes that inherit multiple bases. This is a fairly strict structural design restriction, and more or less makes contract annotations on virtual functions and multiple inheritance incompatible. It will more or less discourage using contract annotations on virtual functions overall to some extent, and certainly discourage using such annotations unless there's some confidence that multiple inheritance mixtures aren't likely to be used.

So, what do we gain?

We don't have to precisely define what "identical" or "has the same list of contract annotations" means. We have fewer technical obstacles to conquer, while keeping the ability to contract-annotate simple single-inheritance cases at their base function level. We can entertain future designs that do not restrict a contract of an override to be identical, we can entertain allowing them to be wider or narrower, once we've had more time to do more design work, more time to think about those problems, more time to discuss them and gain a shared understanding of them, and more time to experiment with those ideas.

We also keep completely open a design option of, in a future revision, allowing a contract on an override to be different from that of the base, being additive. There's a strong likelihood that we will eventually see a proposal

for that.